

Statistical Metrics for Individual Password Strength (Transcript of Discussion)

Joseph Bonneau

University of Cambridge

I'm not proposing any protocols here, I'm talking about passwords, which is what I've spent the last year or so doing now. An interesting problem, which came up in my thesis, is how to tell how strong an individual password is. There's a growing body of publications on how to assess the strength of a big pile of passwords. So if a bunch of passwords leak from a new website there are some measures that I've developed, and some things other people have worked on, to try and compare this new body of passwords to all of the passwords at a different website. But the world of analysing a single password is still in the dark ages I would say. Obviously the difference is that with a group of passwords you can start to do statistics, and you can look at how many passwords are repeated within that set, whereas if you just have one password you have to reason about what set it came from.

The first approach for doing this is to assume some probability distribution that this password was drawn from, then try and figure out roughly what the probability was within that distribution. The traditional way of doing this, the entropy-based method, is that you start looking at the different character classes in the password and the length of it. And then you say, OK, well a password like this, we're going to assume the probability distribution is strings of length 10, which include lower case letters and digits, in which all passwords that are equally likely. Then you can go ahead and do a lot of calculations. This is a screen-shot from a website actually which does this, and it's quite simple. They basically say, we're assuming this comes from an alphabet size of 36 characters, that's 10 characters randomly drawn from that alphabet, so we estimate that this password has a probability 3.76×10^{-15} , and that's basically the strength of this password.

Matt Blaze: And then they add it to their database of passwords.

Reply: Yes. Right, so trusting what goes on over the web, this one actually is in JavaScript . . .

Unknown: JavaScript?

Reply: Yes, so nothing could possibly go wrong. You can run it if you're really paranoid disconnected from the Internet, not that I recommend this site, I can put my cards on the table now and say I think this is really misleading, it doesn't get us very far. In 2005 I believe NIST published some guidelines, the document is called Electronic Authentication Guidelines, and they included this table in the back for how to calculate how strong an individual password is. And this is very simple, you just look at the length of the password, though for some reason they decided to start going by 2s after a while, and then how it was composed, different types of alphabets, whether or not there was a dictionary

check, whether or not it's a member of some known dictionary. And then they give you a quite accurate, with decimal points, estimate for how strong this password is, and this is supposed to be in bits somehow. So we'll come back to this, but the reason I bring this up is that a lot of websites actually base their password strength meter on this table now, and also publications where people look at, say how people choose passwords in different scenarios will actually use this to try and do research on usability.

Frank Stajano: So what are the numbers, when it says 10 characters out of those two columns what do they mean?

Reply: Good question. I'm sure it's in the spec. This table is a little bit confusing.

Frank Stajano: But suddenly when you go to white background you get decimals, and before you get . . .

Reply: Yes. If I'm remembering correctly, this is actually what this space is if you do it randomly, because if it's a ten character alphabet, and you have one character, and it was truly random, then you would have 3.3 bits exactly, right. So these two columns are if the password is drawn completely at random, and these are assuming human choice in different scenarios. So these two columns basically have no information.

Matt Blaze: Well they are made up and the other has guesses.

Reply: Yes, they could be summed up just by one logarithm I guess.

Michael Roe: Going from your previous paper¹, entropy is the wrong measure to be using anyway.

Reply: Yes, right, we'll get to that, but entropy is actually the wrong terminology if you're talking about single events in any case. Entropy is defined over distributions, and if it's a single event you're supposed to say either surprisal or self information, but that has too many syllables so the word entropy gets used in the publication, and sadly it's become a second definition for entropy that people that want to assign to an individual password.

Paul Syverson: Surprisal has the same number of syllables as hedgehog.

Reply: Anyway, there are some interesting proposals for how to model probability distribution of passwords. People have tried to make a Markov model, given the previous N characters, what's the probability distribution of the next character, and you can train this on big datasets. The one that's actually been the most successful when used for training cracking libraries is to model passwords as a probabilistic context-free grammar, so there's some probability that people choose the rule of only use digits, and some other people only use letters, and of course it's recursive because it's a context-free grammar, so it can be a bunch of letters followed by a bunch of digits followed by some more letters, and then you can evaluate all these probabilities on data and get something that's sort of reasonable.

The other approach for trying to assess how strong an individual password is, is to just run some cracking algorithm that you think might actually simulate

¹ The science of guessing: analyzing an anonymized corpus of 70 million passwords, IEEE Symposium on Security and Privacy, Oakland 2012.

your adversary, and see how long it takes to get to the password in question. You could say if the password you're reasoning about is the first thing that the cracking library tries, it's not very good, and if it takes billions and billions of guesses then it starts to seem pretty good. Of course there isn't any standard cracking algorithm, the closest thing now is John the Ripper, which is open source, and people can download it and play with it, but it's still not really standard because it depends on how you configure it. There are different word lists, and there are different mangling rules, which is a problem for research because people will say, we ran John the Ripper and it took this long to crack the password, but you never really know exactly what they ran, so it's actually pretty hard to get precisely the same outcome. And then there's a whole sub universe of proprietary cracking algorithms that you can spend up to \$14,000 on, you can read the text right here, but that's discounted by \$5,000.

Matt Blaze: So there's a standard edition, a forensic edition, and an organised crime edition.

Reply: Yes.

Sandy Clark: I think the organised crime one is the cheapest.

Reply: No, it's the most expensive?

Paul Syverson: You only get to sell it once.

Reply: It's a pretty crazy world. It seems like the main customers for this are actually people who want to spy on their spouse or their teenage kid, or businesses who want to crack their employees' passwords.

Omar Chaudary: What's the main difference between the free version the and the \$14,000 version?

Reply: I don't know.

Paul Syverson: About \$14,000.

Omar Chaudary: Is it 14,000 times faster, or something like that?

Reply: Well actually it's not any faster because it's just software, I mean, it's only as fast as the hardware.

Omar Chaudary: I don't know, perhaps you get a fast implementation?

Reply: To be honest, they don't really provide that much when you go to their website, but maybe they have a slightly more tuned wordlist.

Jonathan Anderson: It seems like the pricing might be meant to target low-information purchases.

Reply: Yes, I don't think that it's dramatically different.

Matt Blaze: How much does their customer list cost?

Reply: I also haven't asked. But there hasn't been any good research that's actually compared them, there's one or two research papers where people have been given the expensive cracking library to try out, but there hasn't been a good comparison yet that actually evaluates this as any better than the thing you can get for free.

Sandy Clark: And standard tools, not the password cracking in particular, but in general, the purchase of the \$3,000 core impact version is much better than the Black Cat 5 which is excellent, but is an order of magnitude better.

Reply: The answer to that is that I really don't know. What I do know is that there are now big datasets of passwords that are fairly easy to come by on the Internet. The RockYou Leak is still the most famous, there were 32 million passwords that got leaked about two years ago. There's a whole basket of others now, nothing as big, but there's a bunch of different datasets with a million passwords or so floating around.

So my approach, particularly for measuring the difficulty of entire distribution against guessing, is to assume that the attacker knows the distribution completely, and then we can reason about this purely as a probability distribution, not have to rely on some complicated model, and not have to rely on cracking software. So, really simply, for this talk the only question I want to address is, if we have a probability distribution that is known completely, and I ask how you evaluate the strength of an individual event against guessing, what's the right way to do this? And I claim that there's two basic properties of any measure for evaluating strength in this model should have, one of which is just that the measure should be uniform for a normal uniform distribution. So if you have a uniform distribution, say random 128-bit strings, every event in that distribution should have the same strength, which should be a log of the space of the distribution. I think that's pretty straightforward, and should at least give us the property that if we do our strength metric on crypto keys it will give the strength. And the other property is also pretty straightforward, just that the strength should be monotonically increasing with less likely events, so the most likely event in the distribution has to be the one that we say is the weakest, and we can only get stronger from there.

Metric number 1, the first thing you might think of, is basically what the entropy metrics I showed you at the beginning are an approximation for, because you just look at the probability of the event you're trying to assign some strength to, which I assume that we know exactly, and you take the logarithm of it. This is exactly the definition that Shannon came up with for surprisal. So the nice things, obviously this is a very simple metric, it does have the uniformity property, it's monotonic. The problem with it is that it really doesn't have anything to do with guessing, there's no direct correlation between this and how difficult something is to guess. It's conceptually nice, but it's not really valid for the problem we're trying to study.

What seems to make a lot more sense is to forget about the probability of the event and just look at what its index is within the distribution. So if it's the most likely event, no matter what its probability is, the fact that it's the most likely means that it's the first thing that an optimal attacker will guess, and we can assign some strength based on that. To get this to have a uniformity property, it's a little bit of a struggle actually, we have to apply a trick, which is to say that if we have a uniform distribution we don't randomly assign index 1 through N to everything in the distribution, but we give everything an index of $\frac{N}{2}$. Basically we don't break ties, if there's a bunch of events that are equally probable we give them all an average index, which is sort of sensible because it says that if there's a bunch of stuff that is equally likely, the adversary won't

always pick the same thing to guess within that set, that the adversary will actually choose randomly, which starts to make it OK.

There's a problem that I get into more detail in my paper about, which isn't super interesting, which is that, if you have an approximately uniform distribution where you can no longer assign everything an average index, but you have some ordering, then all of a sudden the average strength of all the events in that distribution will jump down, so it becomes discontinuous there, which isn't great.

The other problem here is that the strength that you assign to the most likely event in the distribution ends up being zero, which, depending on your philosophy I guess, this is a question for debate, it should be. If you assume that the attacker will guess something first, does that mean that it is useless as a password then? The people who use 123456 which in the real world is almost always the most common password of any population of people, are we comfortable saying that the effective security they get is zero? I don't know.

Omar Chaudary: What is the basis for saying that if you have the set with the same probability you have to give it $\frac{N}{2}$ as a strength, compared to giving it N ?

Reply: I'm not saying $\frac{N}{2}$ is the strength, I'm saying that's the index. If you have a uniform distribution where everything has the same probability, this metric requires that you assign an index to everything, so you have to say where does this rank in the distribution, and you can either break ties arbitrarily and just say we're going to randomly pick something to be the most probable event, and the second most probable, or you can say that everything is the $\frac{N}{2}$ -th most probable event.

Omar Chaudary: Yes, but why $\frac{N}{2}$ and not N for example?

Reply: If you say everything is the least probable event in the distribution, that doesn't make sense either, right?

Omar Chaudary: I'm thinking about the situation where you have $N - 1$ events, all within the same distribution, and one thing which has the least probability. You say the last index is the strongest one, but all the other are just before that somehow?

Reply: Well I say the others should be averaged amongst how many there are.

Omar Chaudary: I'm just asking is there any theory saying that it is better to just say you have them in the middle rather than giving it the maximum strength?

Reply: Well the theory is that, if you don't put it in the middle then the expectation for the uniform distribution won't end up being $\log N$, the uniform distribution will start to look either really bad or really good, because you'll always get the most probable event, or the least probable event. Whereas if you put it in the middle and you say, well I always get an average event, this is basically what happens, right?

Michael Roe: And the intuition is the amount of guesses the attacker has to do before they find it.

Reply: Yes exactly. Right, so the third metric, the third approach I want to take comes out of my thesis work. For my thesis work I developed the philosophy which I think that works the best for assessing the strength of a whole distribution against guessing. You first determine what the adversary's goal is, and parameterize that as α , the proportion of accounts that an attacker is trying to break, and it turns out that an attacker trying to break 50% of accounts versus 10% of accounts, ends up being a really different attacker. There are two metrics you can develop from that, you can either say, well if they want to break 50% of accounts how big does my dictionary have to be, or if they want to break 50% of accounts how much work will I have to do per account. And these are these metrics μ and G , and this graph hopefully will make this make a lot more sense.

Plotted not in logarithmic space, these are some statistics for the distribution of PINs. For an attacker the success rate is increasing from 0 to 1. This blue line is how much bigger your dictionary needs to be. So a fairly small dictionary of under 500 different things will be enough to guess about half of people's PINs correctly. And then eventually to get all the way to 1 your dictionary has to go to size 10,000. That's the dictionary size metric, and the other way is the expected amount of work, which doesn't grow nearly as quickly. The expected amount of work per account if you're going to break every account doesn't get all the way up to 10,000, it gets up to a little over 1,000 guesses per account that you're trying to break.

I think it's a little bit hard to read this graph, in particular this black line, this is what random four-digit PINs would get you, and this is what random three-digit PINs would get you. But I have a trick, which is to convert it to logarithmic space, where it finally starts to make a lot more sense. In logarithmic space both of these metrics are uniform for an attacker with a different success rate. For a random three digit PIN or a random four digit PIN those are equivalent to the attacker, no matter what his desired success rate is. And these other two lines again are pretty close, and the expected number of guesses doesn't get all the way up to the full size of the space. To adapt this to an individual event in an interesting way, let's suppose that my password is 7r1596. If an optimal attacker will guess about 50% of other accounts before getting to that, this is the point in the attack at which the guesser will guess, 7r1596, and we'll say that the expected amount of work the attacker is doing *per account* at that point in the attack, that can be the strength of that password 7r1596.

Jonathan Anderson: Is there a place to plug in more information, such as an attacker who's been to the protocols workshop before, or knows Sydney Sussex, and how they do their codes?

Reply: Yes, so that would mean that the distribution has changed, right? And I guess that's why, for all of the strength metrics I proposed, I've been subscripting by this calligraphy $\mathcal{X}$, which is the distribution that the attacker has assumed. Or it's the distribution that we're assuming when assigning a strength to this password. But I think you fundamentally have to do that because, for 7r1596, there's some distributions in which that's a really, really weak password, like the distribution of wireless passwords in this room where it's on the

white-board. But in some other distribution it might be a really good password, or at least a decent password.

If that transition made any sense of going from the distribution-wide guessing attack to the individual password and saying, at what point does that individual password get caught, this works out really smoothly as a strength metric. The normalisation that monotonicity has already taken care of, and unlike the index metric which assigned a strength of zero to the most probable event, the strength of the most probable event by this metric is the min entropy of the distribution, which to me is a little bit more defensible.

So those are the metrics, and I have some examples here for the RockYou distribution, which is the first place to start when you're evaluating passwords statistically now. Here I have taken, at equally spaced intervals logarithmically in a distribution, some example passwords, this is how many times they occurred in the RockYou distribution, and this is the strength that would get assigned to them by the three metrics I mentioned, probability, index, and this guessing attack metric. And then in this last column, this is the strength of the password you'll get by the NIST formula, which is the table I had at the beginning. I've colour-coded that to compare these four metrics, the red is the biggest overestimate, and the blue is the biggest underestimate of these. The high level trend comparing these, the index metric says that the really common stuff is extremely weak, obviously starting from 0 and then slowly building up. The guessing attack metric is nice, it's usually in-between the other two. Not surprisingly the NIST metric jumps around quite a lot, so it assigns 30 bits to this password, scotty2hotty. I didn't know what it meant, but it turns out it's a professional wrestler, so even though it's fairly long and it has a number in it, 34 people chose that password, and it's not particularly strong, but it does quite well by the NIST metric. Whereas diving, the word diving, looks very bad by the NIST metric because it's short and it just has lower case letters, actually is a dictionary word too, but it's not dramatically more popular than the wrestler's name. I think it's not surprising that the NIST metric doesn't do particularly well head to head against the other ones.

As a slightly more interesting experiment, I took a bunch of datasets of passwords that are too small to actually do statistics on. If you only have 200 passwords you can't start to reason about the distribution, but what you can do is you can look at how strong each of those passwords would be if you assume that they were drawn from the RockYou distribution, and try and assign a strength to that set of passwords by averaging. One reason people really want to do this is to do usability experiments where you bring in sets of users and you add two different conditions, maybe one set of users you give a really strong policy to, the other users you just say, you can choose passwords however you want. Maybe you can only do 100 users this way, but you want to ask the question of which group of users actually picked better passwords, and how did it change.

Matt Blaze: RockYou was from a single service, right, that was compromised.

Reply: Yes.

Matt Blaze: Not from multiple services? So it was fairly homogeneous?

Reply: Yes and no. It's a company that develops a bunch of different social applications. They have a couple of different Facebook games, some photo sharing applications, and they run one sign-on service for all the different applications they develop. But essentially, yes, it's mostly Facebook users, most of the passwords are English speakers, though there are a lot of Spanish speakers too. But it's not like when I collected some data at Yahoo, which is really nice, as they run a little bit of everything.

Right, so that's really a scenario where you would want to apply the individual strength metrics to a whole group of passwords, if you have a small sample and you're trying to figure out how two different groups compare, and this is what the NIST metric gets used quite a lot for in the password usability research now. I wanted to see if these statistical metrics I proposed look any better or worse.

I think that there's actually definitely some cases where the NIST formula produces more plausible results. I had one dataset of passwords chosen by Chinese speakers, and one dataset of passwords chosen by Hebrew speakers, which look relatively much stronger by the statistical metrics if you use RockYou as the baseline, because RockYou didn't have very many Chinese or Hebrew speakers, so the stuff that they pick looks relatively strong if you put it into the RockYou distribution. But by the NIST formula those passwords get scored much more weakly because the NIST formula doesn't have any concept of language or semantics, and so if they're short and they don't have numbers then they're rated weakly.

Frank Stajano: When you're doing Chinese does it mean Chinese with Chinese characters?

Reply: Well in practice no, because Chinese users almost never actually use Chinese characters in their passwords.

Frank Stajano: But if you did wouldn't the NIST formula have to be modified so each letter doesn't account for 26?

Reply: Yes, the NIST formula is really only defined for ASCII, but at least in the datasets that Rubin and I have been looking at, Chinese speakers essentially never use Chinese characters in passwords, which is partly an artefact of the way that they input the characters.

Frank Stajano: I was just wondering if this thing could be even plugged into the NIST formula if it contained Chinese characters?

Reply: I suppose, if you treated each Chinese character as one letter. But then, it would produce strange results because the alphabet is so much bigger. The NIST algorithm doesn't have instructions on how to deal with that problem.

Hannan Xiao: I use Pinyin for the Chinese characters.

Reply: Yes, exactly. I think that the cases where the NIST formula does pretty poorly, down here I have dictionaries which are sort of cheating because they're chosen to be very weak passwords, either the passwords that are contained in different Malware that tries to do guessing attacks, or the blacklist that Twitter uses, which they for a long time now have implemented client-side in JavaScript. I check it every month or so and see how they update the blacklisted

passwords that they don't allow. It was actually a terrible blacklist too up until about a year ago, it had a bunch of really uncommon names in it, and it didn't have 123456 for a long time, which is kind of amazing because it's always the most common password. I think that actually the statistical metrics capture this really well, they say that the passwords on the 2010 blacklist were actually quite strong, which is a bad thing if you're developing a blacklist. And then in 2011 they finally did a more sensible one, and the average strength of these passwords went way down, but by the NIST formula there was basically no difference. So that was the case where the NIST formula really struggled.

OK, there's a lot more about the math and how to estimate these things from samples, and deal with unseen events and things like that in my paper and my thesis, but at a high level this covers what I wanted to propose today.